

Generics and Collections

Universal (Generic) Algorithms

When using generics in Java, universal (generic) algorithms can be created for different types.

Generics

In Java, generics allow classes and interface types to be used as parameters to define classes, interfaces, or methods.

Benefits of Generics

In Java, generics allow for stronger type checking and bug detection at compile time.

Diamond Operators

When using generics in Java, the diamond operator (`<>`) is used to declare the type parameter.

super

When using generics in Java, the `super` keyword is used to define a lower bound type on a wildcard.

Wildcards

In Java, the wildcard (`?`) is used to specify an unknown generic type parameter.

extends

When using generics in Java, the `extends` keyword is used to define an upper bound type on type parameter or wildcard.

Wrapper Classes

Wrapper classes are provided to allow primitive values to be used with generic code.

Collections

A collection stores many elements within a single object.

Collections Framework

The collections framework provides ready to use collection data structures and algorithms.

Aggregate Operations

When using collections in Java, aggregate operations are used to access and modify a stream of elements.

Collection Interface

The `Collection` interface is the root interface of the collections framework.

List Interface

The `List` interface is used to store elements in an ordered collection.

Set Interface

The `Set` interface is used to store unique elements in a collection.

Queue and Deque Interfaces

The `Queue` and `Deque` interfaces are used to hold elements in a collection to process in a specific order.

Map Interface

The `Map` interface stores elements as key-value pairs.

Iterators and Enhanced For Loops

Iterators and enhanced `for` loops are used to traverse collections in Java.

Type Parameters

Collections are generic, and a type parameter must be specified.